

```
import cv2
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from google.colab.patches import cv2_imshow
```

Task 1

Task 1.1: Filter Exploration

```
img = cv2.imread('/content/drive/MyDrive/imgDataset/devfest.png')
```

```
image = cv2.resize(cv2.cvtColor(img, cv2.COLOR_BGR2RGB), (256,256))
cv2_imshow(image)
```



```
img_gray = cv2.resize(cv2.cvtColor(image, cv2.COLOR_RGB2GRAY), (256,256))
cv2_imshow(img_gray)
```



```
# Gaussian blur
gaussian = cv2.GaussianBlur(img_gray, (15,15), 0, )
cv2_imshow(gaussian)
```



```
# Canny edge detection
canny = cv2.Canny(img_gray, 10, 50)
```

```
cv2.imshow('canny')
```



```
# Laplacian edge detection
laplacian = cv2.Laplacian(img_gray, cv2.CV_64F)
cv2.imshow('laplacian')
```



```
# original and three filtered versions in a 2x2 grid
plt.figure(figsize=(10,8))

plt.subplot(2,2,1)
plt.imshow(image)
plt.title("Original")
plt.axis("off")

plt.subplot(2,2,2)
plt.imshow(gaussian, cmap="gray")
plt.title("Gaussian Blur")
plt.axis("off")

plt.subplot(2,2,3)
plt.imshow(canny, cmap="gray")
plt.title("Canny Edge")
plt.axis("off")

plt.subplot(2,2,4)
plt.imshow(laplacian, cmap="gray")
plt.title("Laplacian")
plt.axis("off")

plt.tight_layout()
plt.show()
```

Original



Gaussian Blur



Canny Edge



Laplacian



```
# about filters
# Gaussian Blur: Smooths the image and reduces noise.

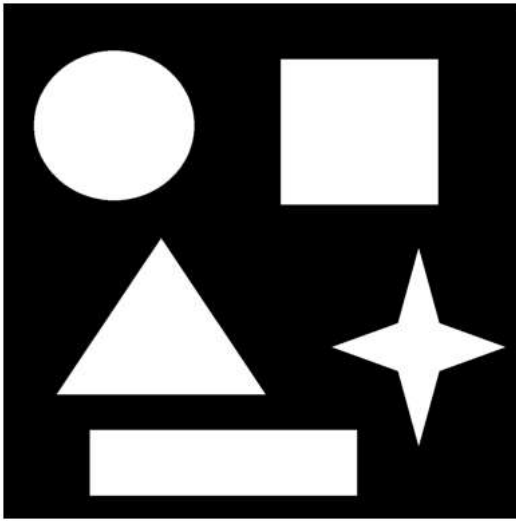
# Canny Edge Detection: Detects strong edges and object boundaries.

# Laplacian: Detects edges in all directions and highlights image details.
```

Task 1.2: Simple Pattern Detection

```
# simple black and white image
img_draw = cv2.imread('/content/drive/MyDrive/imgDataset/simple.png')
```

```
plt.figure(figsize=(5,5))
plt.imshow(img_draw)
plt.axis("off")
plt.show()
```



```
# Applying Canny Edges
edges = cv2.Canny(img_draw,50,150)
```

```
#Compare result with original image
```

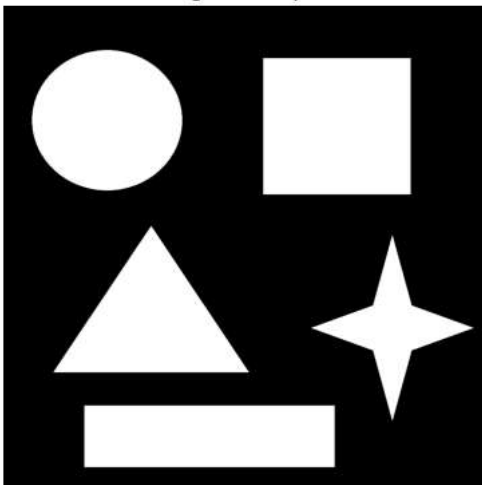
```
plt.figure(figsize=(8,4))

plt.subplot(1,2,1)
plt.imshow(img_draw,cmap="gray")
plt.title("Original Shapes")
plt.axis("off")

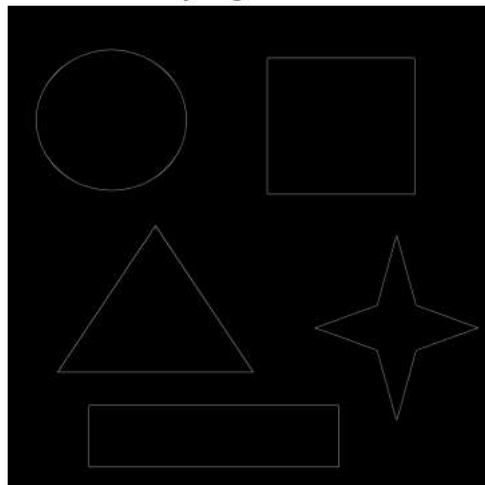
plt.subplot(1,2,2)
plt.imshow(edges,cmap="gray")
plt.title("Canny Edge Detection")
plt.axis("off")

plt.tight_layout()
plt.show()
```

Original Shapes



Canny Edge Detection



Task 2

Task 2.1: Manual Convolution Practice

```
## Perform Manual Convolution
def simple_convolution ( image , kernel ) :
    """"
```

```

Simple convolution implementation for learning purposes
"""

# Get image and kernel dimensions
img_height , img_width = image . shape
kernel_height , kernel_width = kernel . shape

# Calculate output size ( without padding )
output_height = img_height - kernel_height + 1
output_width = img_width - kernel_width + 1

# Initialize output
output = np . zeros (( output_height , output_width ) )
for i in range ( output_height ) :
    for j in range ( output_width ) :
        # Extract the region
        region = image [ i : i + kernel_height , j : j + kernel_width ]
        # Element - wise multiplication and sum
        output [ i , j ] = np . sum( region * kernel )

return output

```

```

# small test image
image = np.array([
[1,1,1,0,0],
[0,1,1,1,0],
[0,0,1,1,1],
[0,0,1,1,0],
[0,1,1,0,0]
], dtype=np.float32)

```

```

# edge detection kernel
kernel = np.array([
[-1,-1,-1],
[-1,8,-1],
[-1,-1,-1]
], dtype=np.float32)

```

```

# Manual Filter
manual = simple_convolution(image, kernel)
manual

```

```

array([[ 3.,  2.,  3.],
       [-4.,  2.,  2.],
       [-4.,  3.,  3.]])

```

```

# Opecv Filer
opencv = cv2.filter2D(image,-1,kernel)
opencv

```

```

array([[ 2.,  2.,  1., -5., -4.],
       [-5.,  3.,  2.,  3., -5.],
       [-2., -4.,  2.,  2.,  2.],
       [-2., -4.,  3.,  3., -5.],
       [-2.,  5.,  3., -5., -4.]], dtype=float32)

```

```

# Compare your manual result
plt.figure(figsize=(10,3))

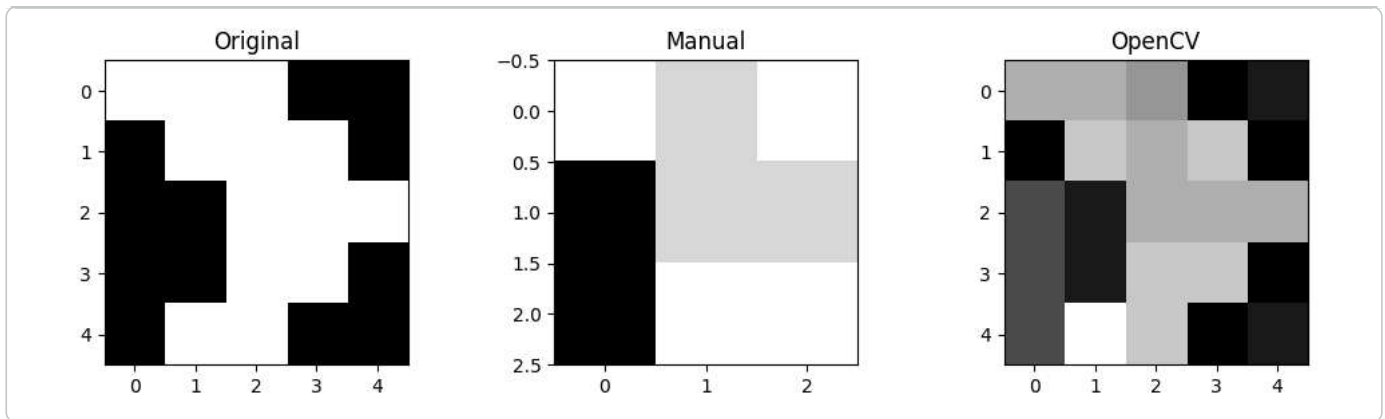
plt.subplot(1,3,1)
plt.imshow(image,cmap="gray")
plt.title("Original")

plt.subplot(1,3,2)
plt.imshow(manual,cmap="gray")
plt.title("Manual")

plt.subplot(1,3,3)
plt.imshow(opencv,cmap="gray")
plt.title("OpenCV")

plt.tight_layout()
plt.show()

```



Task 2.2: Kernel Experiment

```
image = cv2.imread('/content/drive/MyDrive/imgDataset/fruits.png')
```

```
image_1 = cv2.resize(cv2.cvtColor(image, cv2.COLOR_BGR2RGB), (256,256))
cv2.imshow(image_1)
```



```
# Kernels
vertical = np.array([
    [-1,2,-1],
    [-1,2,-1],
    [-1,2,-1]
])
```

```
horizontal = np.array([
    [-1,-1,-1],
    [2,2,2],
    [-1,-1,-1]
])
```

```
sharpen = np.array([
    [0,-1,0],
    [-1,5,-1],
    [0,-1,0]
])
```

```
# Apply Vertical, Horizontal and Sharpen Kernels
v = cv2.filter2D(image,-1,vertical)
h = cv2.filter2D(image,-1,horizontal)
s = cv2.filter2D(image,-1,sharpen)
```

```
# plot Results
plt.figure(figsize=(10,8))

plt.subplot(2,2,1)
plt.imshow(image_1,cmap="gray")
plt.title("Original")
plt.axis("off")

plt.subplot(2,2,2)
```

```

plt.imshow(v,cmap="gray")
plt.title("Vertical Kernel")
plt.axis("off")

plt.subplot(2,2,3)
plt.imshow(h,cmap="gray")
plt.title("Horizontal Kernel")
plt.axis("off")

plt.subplot(2,2,4)
plt.imshow(s,cmap="gray")
plt.title("Sharpen Kernel")
plt.axis("off")

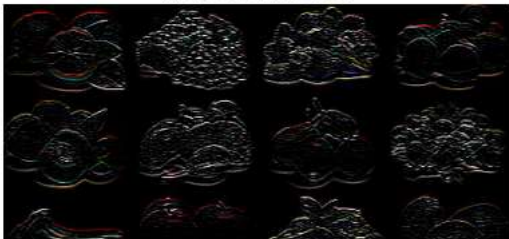
plt.tight_layout()
plt.show()

```

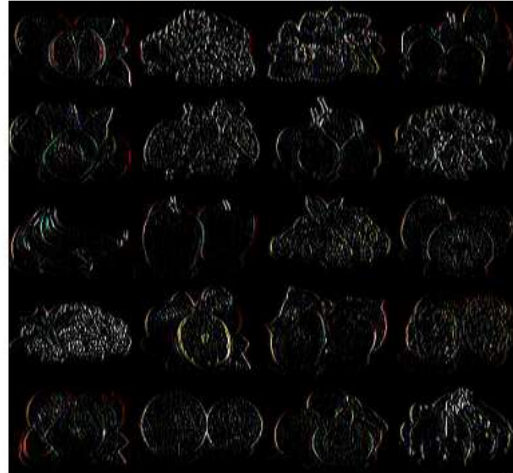
Original



Horizontal Kernel



Vertical Kernel



Sharpen Kernel

